

Expiry Date Capture System

Proposed architecture for backfilling missing expiry dates in warehouse inventory

Version 1 scope ·

1. The problem

The inventory database holds product ID, batch number and other master data, but no expiry date. The barcode printed on the back of each pack does not encode expiry either, so the date exists only as printed text on the physical packaging. Until that text is digitised, the business cannot run FEFO picking, expiry alerts, or write-off forecasting on existing stock.

The proposal below uses idle warehouse staff time to capture that data with a mobile web app: one photograph per batch, read automatically, confirmed with a single tap.

2. Three decisions that shape the design

2.1 Expiry is batch-level data, not unit-level

Every unit manufactured in the same batch carries the same expiry date. Capturing per unit repeats the same work hundreds of times; capturing per unique SKU + batch combination captures it once and propagates it to every unit linked to that batch.

Approach	Photos needed	Time at 8s each	Result
Per physical unit	~20,000	~44 hours	Repetitive, expensive
Per SKU + batch	~1,200	~2.7 hours	Same coverage, far cheaper

Illustrative figures based on 20,000 units, 600 SKUs and roughly 1,200 unique SKU + batch combinations. Confirm the real batch count before committing to a timeline.

2.2 One photograph, two extractors

The back of the pack usually shows the barcode and the printed expiry in the same frame. So the employee performs a single action - take one photo - and the system runs two independent extractors on that same image: a barcode decoder to identify the product, and OCR to read the batch number and expiry date. This removes the separate scanning step entirely.

Important: decode the barcode itself rather than OCR-ing the digits printed under it. OCR routinely confuses 0/O, 1/I, 5/S and 8/B, whereas a barcode carries its own check digit and either decodes correctly or not at all.

2.3 The machine reads, the human confirms

A silent write is dangerous: a single misread of 08/2027 as 08/2021 would mark an entire batch as expired. The employee therefore sees a one-line summary and taps once to accept. No typing is involved, so the workflow stays fast while accuracy stays high. Once measured accuracy justifies it, high-confidence reads can be auto-saved and only uncertain ones sent for review.

3. System architecture

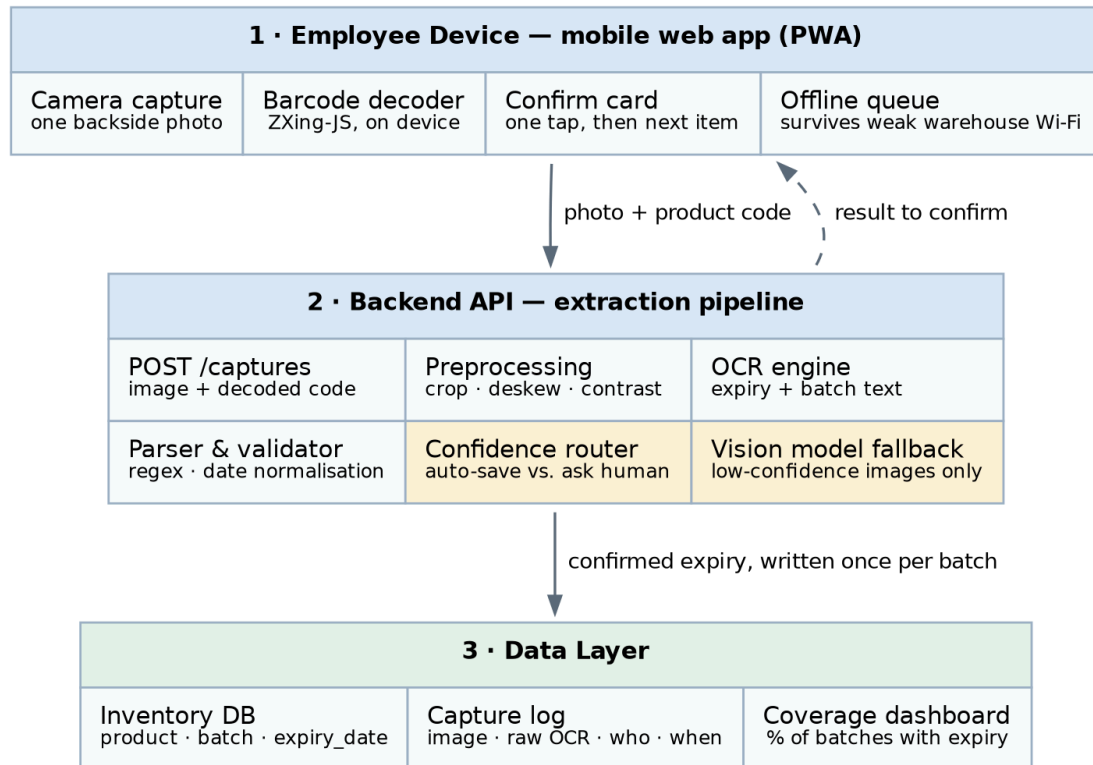


Figure 1 — Three-layer architecture: capture on the device, extraction in the API, expiry stored once per batch.

Mermaid source for all three figures is in Appendix A, ready to paste into any Mermaid-capable tool.

4. Capture pipeline

What happens between the shutter press and the database write:

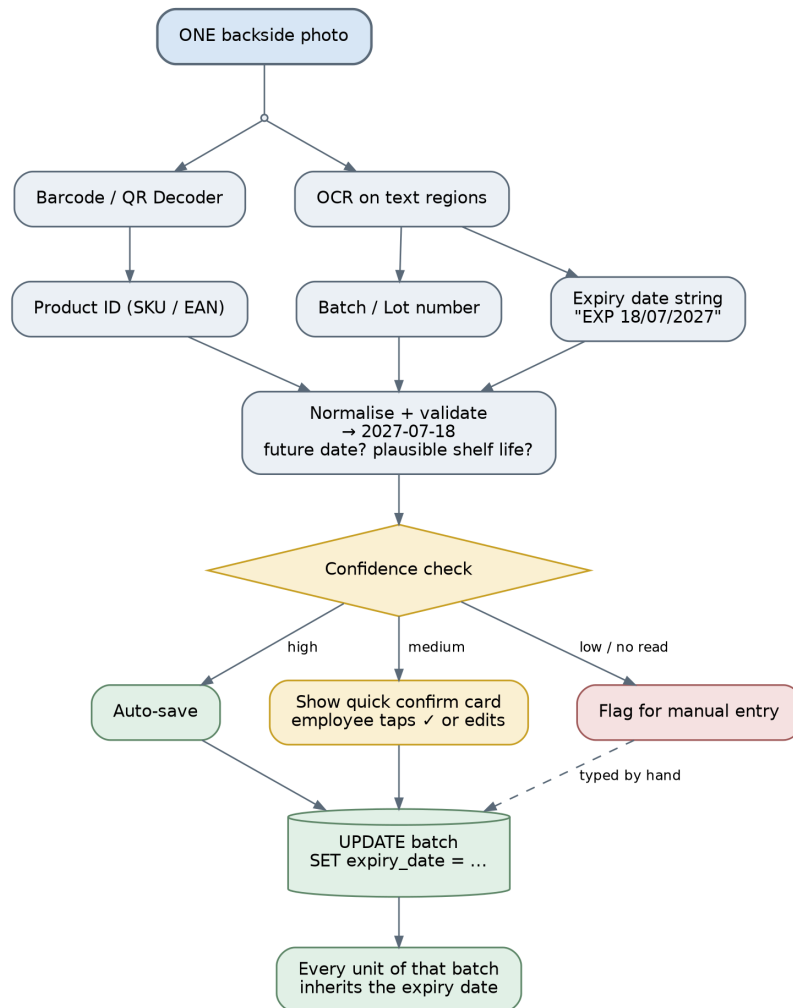


Figure 2 — One photo in, one confirmed batch expiry out.

5. Data model

The single most important structural change: expiry_date belongs to the batch table, and inventory units inherit it through their batch link. The capture log keeps the original image and raw OCR text so any value can be audited or re-processed later without a second warehouse pass.

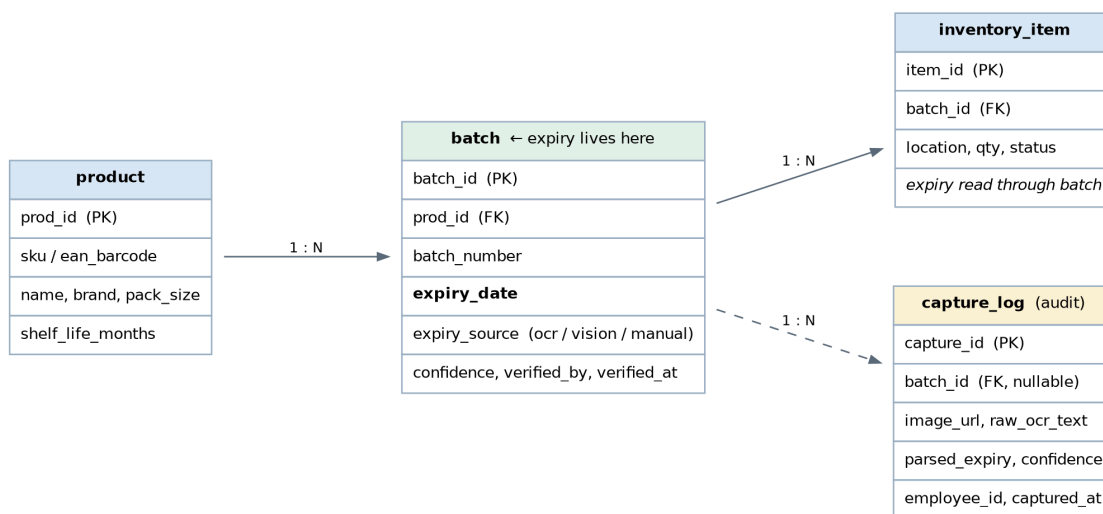


Figure 3 — Expiry stored once on the batch; every unit reads through it.

6. Component choices for version 1

Layer	Recommended for v1	Notes
Client	Mobile web app (PWA), camera via getUserMedia	No app store, works on the phones staff already carry; add to home screen
Barcode	ZXing-JS in the browser; ZBar / pyzbar or OpenCV server-side	Decode on device where possible so a failed read is caught before upload
Preprocessing	OpenCV: crop, grayscale, deskew, adaptive threshold	Biggest single lever on OCR accuracy, especially for dot-matrix ink codes
OCR	Tesseract (restricted charset) or a hosted OCR API	Constrain to digits and separators; try a couple of preprocessing variants
Parsing	Regex + date library, with validation rules	Handle DD/MM/YYYY, MM/YYYY, MM-YY, and MFG date plus shelf life
Fallback	Vision-language model on low-confidence images only	Keeps cost low; used for a minority of captures
Storage	Existing inventory DB + object storage for images	No new database needed; add batch expiry columns and a capture log table

Do not train a custom computer-vision model at this stage. A barcode decoder, solid preprocessing, OCR and strict validation are enough for version 1, and the capture log will provide labelled training data if a custom model is ever justified.

7. Confidence policy

Confidence	Condition	Action
High	Barcode decoded, date parses, passes sanity rules	Auto-save (after accuracy is proven in pilot)
Medium	Date parses but OCR score is moderate or format is unusual	Show confirm card; employee taps to accept or corrects
Low	No barcode, unreadable date, or implausible value	Flag for manual entry; keep image for review

Sanity rules worth enforcing: the date must be in the future or recently past, within the product's known shelf life, and consistent with other captures of the same batch. Two captures of one batch that disagree should raise a flag rather than overwrite silently.

8. Suggested rollout

Phase	Scope	Goal
Phase 1	Capture + confirm, one photo, manual fallback	Prove accuracy on 200-300 real packs; measure read rate per supplier
Phase 2	Preprocessing tuning, batch propagation, coverage dashboard	Clear the legacy backlog; track % of batches with a verified expiry
Phase 3	Confidence thresholds, auto-save, vision fallback	Cut confirmations to the uncertain minority
Phase 4	Capture at goods-in, FEFO picking and expiry alerts	Stop the gap recurring for new stock

9. Open questions to confirm before build

- What exactly is the number printed below the barcode - SKU, batch/lot, or a per-unit serial? This determines whether the barcode alone is enough to identify the batch.
- Do all units of a batch genuinely share one expiry date, or do some suppliers vary it within a batch?
- Which date formats appear across suppliers, and do some packs print only a manufacture date plus shelf life?
- How is the code printed - laser, thermal, or dot-matrix inkjet? Dot-matrix on a curved or reflective pack is the hardest case for OCR and should be in the pilot sample.
- Can staff photograph in the aisle with adequate light, or is a fixed capture station with consistent lighting preferable for the backlog?
- Is warehouse Wi-Fi reliable enough for immediate upload, or must the offline queue be in scope from day one?

